

INFORME FINAL – DESAFÍO 1

Implementación de Sweet Crush mediante manipulación de bits en C++

Curso: Informática II

Semestre: 2026-2

Alejandro Macias Rodríguez

CONTEXTUALIZACIÓN:

En el informe preliminar planteé el desarrollo de una versión de Sweet Crush utilizando operaciones a nivel de bits. El reto principal fue representar un tablero rectangular sin recurrir a una matriz convencional de una celda por ficha, decidí almacenar todas las fichas en una secuencia compacta de bits, administrada mediante memoria dinámica.

El juego utiliza seis tipos de fichas (A, B, C, D, E y F). Como tres bits permiten representar ocho valores, reservé los códigos 0 a 5 para las fichas, el 6 para una posición vacía y el 7 como valor inválido o reservado para futuras versiones del juego. Además de formar y eliminar combinaciones horizontales y verticales, el programa permite aplicar gravedad, generar fichas de reemplazo, resolver cascadas y modificar el número de filas y columnas en cualquier posición.

ANÁLISIS DEL PROBLEMA Y CONSIDERACIONES PARA LA SOLUCIÓN PROPUESTA:

Desde el comienzo identifiqué que debía distinguir el tablero que ve el jugador del bloque físico que guarda la información. Una ficha necesita tres bits, no un byte completo. Para un tablero de F filas y C columnas, calculé la cantidad mínima de bytes mediante:
$$(F * C * 3 + 7) / 8$$

Este cálculo redondea hacia arriba cuando los bits no completan el último byte. Por ejemplo, un tablero de 5×5 contiene 25 fichas: $25 \times 3 = 75$ bits y necesita 10 bytes. Con esta decisión mantuve un arreglo dinámico de unsigned char como almacenamiento físico, sin tratar cada uno de sus bytes como si fuera una ficha independiente.

Otro problema fue pasar de una posición (fila, columna) a los tres bits correspondientes. Aunque realmente quedó bien explicado en la guía del desafío entonces casi que fue solo implementarlo (no creo que valga la pena volver a escribir toda la lógica de esta operación en este informe debido a que ya está explícito no solo en la guía sino también en mi informe preliminar el proceso de pasar de la posición lógica a su correspondiente ficha, sin embargo, al final de este informe vuelvo a adjuntar el análisis en mi cuaderno).

Si una ficha comienza en el bit 6 o 7, parte de ella queda en el byte siguiente. Por eso necesité combinar temporalmente ambos bytes en una variable más amplia, desplazar los bits y aislar los tres que correspondían a la ficha.

También debía encontrar combinaciones de tres o más fichas iguales, tanto horizontal como verticalmente. Para esto tuve en cuenta que una misma ficha podía pertenecer a dos combinaciones al mismo tiempo, por lo que debía evitar contarla dos veces. Además, después de eliminar las combinaciones, las fichas restantes debían caer y los espacios vacíos rellenarse. Como esto podía generar nuevas combinaciones, era necesario repetir el proceso hasta que no quedaran más.

Finalmente, para agregar o eliminar una fila o columna en cualquier posición, decidí crear temporalmente un nuevo tablero y copiar las fichas que debían conservarse. Esto fue

necesario porque el tablero está almacenado como una secuencia de bits y no como una matriz normal. Además, llevé un control de los bytes necesarios y los bytes reservados para administrar correctamente la memoria.

A continuación, adjunto una tabla que describe como organicé el proyecto por etapas:

Etapas	Tarea que desarrollé	Resultado
1. Base de memoria	Calcular bytes, reservar/liberar memoria, leer y escribir tres bits.	Tablero compacto.
2. Inicialización	Generar valores 0-5 y mostrarlos como A-F.	Tablero visible y aleatorio.
3. Reglas	Buscar combinaciones horizontales y verticales y marcarlas como vacías.	Posiciones con valor 6.
4. Reorganización	Aplicar gravedad, rellenar y repetir combinaciones.	Tablero estable después de las cascadas.
5. Estructura dinámica	Agregar/eliminar filas y columnas; controlar el bloque reservado.	Cambio de dimensiones y capacidad.
6. Integración	Incorporar menú, entradas, contadores, puntos y visualización binaria.	Partida interactiva hasta la meta.

ALGORITMOS IMPLEMENTADOS:

Implementé las funciones crearTablero(), obtenerFicha() y establecerFicha() para reservar memoria y almacenar o consultar las fichas utilizando 3 bits por cada una. Para esto utilicé operaciones bitwise que permiten modificar los bits correspondientes a una ficha sin afectar las demás, incluso cuando esta se encuentra repartida entre dos bytes.

Luego implementé funciones para llenar el tablero con fichas aleatorias entre los valores 0 y 5, que representan las letras A-F. También desarrollé la visualización del tablero tanto con letras como en formato binario, permitiendo observar la representación de cada ficha mediante sus 3 bits.

Implementé eliminarCombinaciones(), que recorre las filas y columnas buscando grupos de tres o más fichas iguales. Para evitar problemas cuando una ficha pertenece a dos combinaciones al mismo tiempo, utilicé una copia temporal del tablero para realizar la búsqueda. Las fichas encontradas se eliminan utilizando el valor 6, que representa una posición vacía.

Desarrollé aplicarGravedad() para hacer que las fichas caigan y ocupen los espacios vacíos. Después, rellenarVacíos() genera nuevas fichas para completar el tablero.

Finalmente, resolverCascadas() repite estos procesos mientras sigan apareciendo combinaciones, hasta que el tablero quede estable.

Implementé funciones para agregar o eliminar filas y columnas en cualquier posición. Para hacerlo, construyo temporalmente un nuevo tablero y copio las fichas que deben conservarse.

También incorporé el control de memoria mediante bytesReservados, aplicando la regla del 65 % para determinar cuándo es necesario reducir el bloque de memoria. Si al agregar filas o columnas la memoria disponible no alcanza, se reserva un bloque más grande.

Finalmente, integré todas las funciones en un menú interactivo donde el usuario puede eliminar fichas, agregar o eliminar filas y columnas.

También implementé el conteo de fichas eliminadas, combinaciones y cascadas, junto con un sistema de puntuación que otorga 10 puntos por ficha eliminada y 20 puntos adicionales por cascada. El juego termina cuando se alcanzan o superan los 300 puntos.

PROBLEMAS QUE AFRONTÉ:

En la siguiente tabla resumo las principales dificultades que enfrenté durante el desarrollo y cómo las solucioné:

Dificultad	Cómo la resolví
Una ficha quedaba dividida entre dos bytes.	Leí ambos bytes en un bloque temporal, apliqué desplazamientos y una máscara de tres bits.
Una ficha podía cruzar dos combinaciones.	Analicé horizontales y verticales sobre una copia compacta y conté solo posiciones borradas una vez.
Después de eliminar quedaban huecos dispersos.	Separé la caída de fichas del relleno aleatorio y repetí el proceso con resolverCascadas().
Insertar o quitar columnas cambiaba la posición de todos los bits.	Reconstruí un nuevo tablero lógico a través de obtenerFicha() y establecerFicha().
La memoria podía quedar sobredimensionada tras varias eliminaciones.	Distinguí bytes necesarios de reservados y apliqué la condición estricta del 65 %.

EVOLUCIÓN DE LA SOLUCIÓN Y CONSIDERACIONES PARA TENER EN CUENTA EN LA IMPLEMENTACIÓN:

El progreso del proyecto quedó documentado en GitHub mediante cinco avances y una versión final. En la siguiente tabla resumo lo que desarrollé en cada etapa:

Versión	Cambio principal
Avance_1	Reservé memoria y programé obtenerFicha() y establecerFicha() con tres bits por ficha.
Avance_2	Automaticé el llenado y la visualización de fichas A-F.
Avance_3	Incorporé combinaciones horizontales y verticales y el valor 6 como vacío.
Avance_4	Agregué gravedad, relleno y cascadas automáticas.
Avance_5	Agrague funciones para eliminar fichas, agregar o eliminar filas y columnas en cualquier posición y control del 65 %.
Versión final	Integré menú, validaciones, visualización binaria, contadores, puntuación y meta de 300 puntos.

Durante la implementación mantuve seis fichas jugables, un código para vacío y uno reservado.

PRUEBAS Y RESULTADOS:

En la ejecución mostrada del Avance_4 observé un tablero de 5×5 que resolvió una ronda de cascadas, volvió a llenarse y reportó 10 bytes de almacenamiento necesario. En la ejecución del Avance_5 observé la siguiente evolución al eliminar una fila y una columna internas y después agregarlas nuevamente:

Estado del tablero	Bytes necesarios	Bytes reservados
5×5 inicial	10	10
4×5 tras eliminar fila 2	8	10
4×4 tras eliminar columna 2	6	6
5×5 tras agregar fila y columna	10	10

Esta secuencia permite verificar la lógica esperada: 8/10 representa un uso del 80 % y no reduce capacidad; 6/10 representa un uso del 60 % y sí provoca la reducción a 6 bytes. Cuando agregué nuevamente estructura, la capacidad volvió a 10 bytes. La consola terminó normalmente en las ejecuciones mostradas.

La versión final incorpora además las cinco opciones en el menú, muestra cada ficha con su código binario, contabiliza las fichas borradas y las cascadas y utiliza la meta de 300 puntos. Las salidas concretas varían entre partidas porque las fichas se generan pseudoaleatoriamente.

CONCLUSIONES:

Con este desafío logré convertir la idea del informe preliminar en un juego funcional organizado por módulos. El mayor aprendizaje fue comprender que la representación que ve el usuario no tiene por qué coincidir con la forma en que guardo los datos: pude manipular un tablero de filas y columnas sobre una tira continua de fichas de tres bits.

También comprobé como nunca lo había hecho la importancia de separar responsabilidades. Ahora entiendo por qué es que tanto nos dicen: “divide y vencerás”. En este caso las funciones de lectura y escritura permitieron construir combinaciones, gravedad y cambios de dimensiones sin repetir operaciones en cada caso. El uso de una copia compacta me ayudó a resolver coincidencias simultáneas y a contabilizar correctamente las fichas eliminadas.

El seguimiento de memoria necesaria y reservada me permitió integrar la regla del 65 % de forma visible. Los avances documentados en GitHub muestran cómo pasé de probar funciones aisladas a reunir las en una partida interactiva con puntuación y condición de finalización.

Finalmente, y hablando ya desde el ámbito personal, aprendí que un programa no se construye de una sola vez, sino que requiere analizar, probar, corregir y mejorar constantemente. Durante el desarrollo tuve diversas dificultades, sin embargo, una muy concreta estuvo cerca de hacerme cancelar el curso, pero decidí seguir adelante. Es precisamente esto lo que debemos hacer cuando tenemos en frente un problema como lo fue este desafío en frente, tal vez no sepamos no solo como empezar sino absolutamente

nada sobre qué debamos hacer, pero es entonces que simplemente debemos seguir adelante, intentarlo de la mejor manera posible, con mucha actitud y carácter, para finalmente analizar los resultados con el fin de siempre querer mejorar.

Para finalizar y como anexo deo también adjuntadas las imágenes incluidas en el informe preliminar:

Tamaño del tablero en memoria para un tablero $F \times C$
 existen $F \times C$ - posiciones, la cantidad mínima de
 bits sera $F \times C \times 3 \rightarrow$ bytes: $3 \times F \times C / 8$ redondeado hacia arriba

La memoria reservada debera corresponder a las
 dimensiones **ACTUALES** del tablero

tablero conceptual

	0	1	2	3	Columnas
0					
1					
2					
3					

(Fila, columna) = (2,3) Posición lógica

Indice: ¿Que ficha es? (0,1,2,3,4)

Fila $\times$ Columnas + Columna = 77

Bit inicial: Índice $\times 3 = 51$

Ahora queremos saber en que byte esta:

Sabemos que:

Byte 0: 0 - 7	$\rightarrow$ aquí esta nuestra ficha $byte = bit\ inicial / 3 = 6$ $posicion\ en\ ese\ byte = 51 \% 8 = 3$ estan en el byte 6 en el bit 3
Byte 1: 8 - 15	
Byte 2: 16 - 23	
Byte 3: 24 - 31	
Byte 4: 32 - 39	
Byte 5: 40 - 47	
Byte 6: 48 - 55	

Posición lógica $\rightarrow$ Índice $\rightarrow$ Bit inicial $\rightarrow$ Byte correspondiente

↓
Bits de la ficha

Flujo: el usuario establece la dimension

↓
Reserva de memoria

↓
Menu: eliminar ficha, fila o columna ←

↓
generar nuevas ficha

↓
deteccion nuevas combinaciones

↓
eliminar combinaciones

↓
reorganizar tablero

↓
Si — ¿Existen mas combinaciones? — No